Projet Linux : Application Web multi-services

ABDALLAH Maïssane

3IWJ

Contexte du projet Multi Webapp

Dans le cadre de ce projet réalisé à l'ESGI, le but était de développer une application web multi-services capable de gérer et d'afficher des articles en temps réel.

Ce projet intègre de plusieurs outils côté serveur et client. L'objectif principal était de mettre en œuvre une architecture complète combinant une API REST avec Flask, un serveur WebSocket en Go pour la gestion en temps réel, un proxy inverse avec Nginx et une interface front-end en HTML/JavaScript, le tout soutenu par une base de données SQLite pour assurer la sauvegarde des données.

Enfin, un système de services systemd permet de gérer le déploiement et le démarrage automatique des différents services nécessaires au bon fonctionnement de l'application.

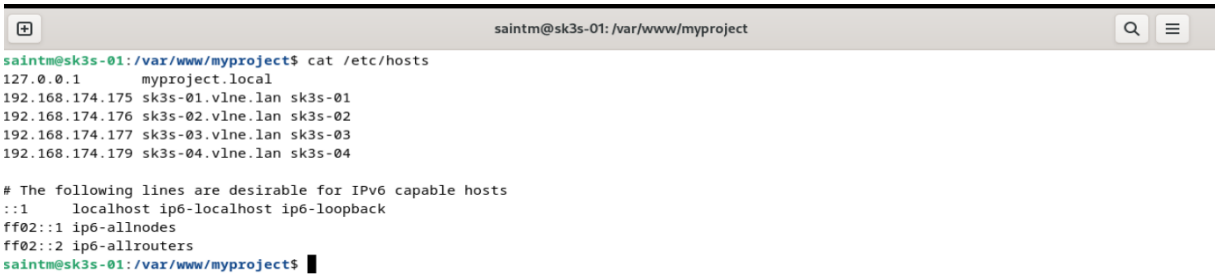
L'application finale permet :

- L'affichage d'une liste d'articles,
- L'ajout d'article depuis le serveur,
- La mise à jour de cette liste en temps réel via WebSocket, avec envoie de notifications dans la console et dans le service,
- Le routage des requêtes grâce au server proxy Nginx.

Technologies utilisées

Le Projet repose sur une architecture combinant plusieurs technologies orientées back-end et front-end afin d'assurer un fonctionnement fluide, en temps réel et évolutif.

J'ai changé le domaine local (localhost) de ma VM dans le fichier /etc/hosts afin de le renommer 'myproject.local'.



Voici les technologies principales utilisées dans ce projet :

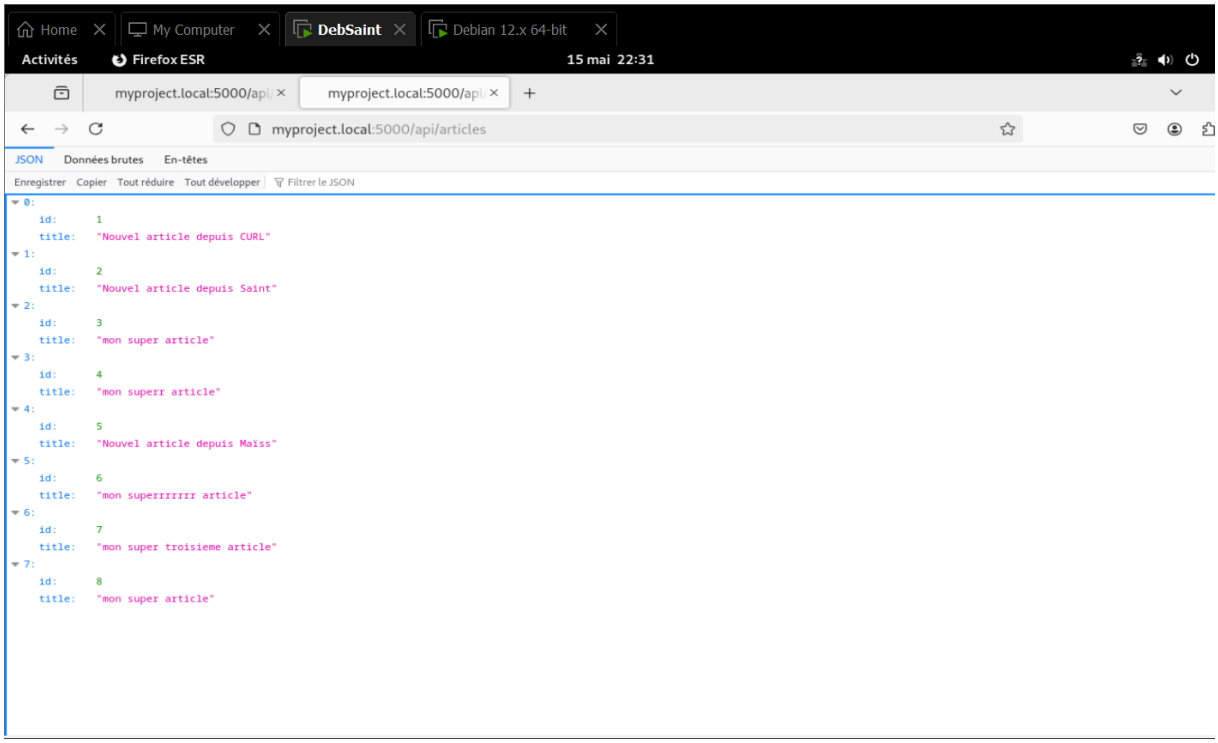
1. Le Framework Flask (Python)

Flask est un Framework utilisé pour développer des application web et des API REST. Il permet de gérer les routes HTTP (GET, POST) de l'application, structurer une API rapide à déployer et d'intégrer une base de données SQLite via Flask-SQLAlchemy. L'API permet de consulter la liste des articles existants et d'en ajouter de nouveaux

J'ai configuré un environnement virtuel, l'ai activé, puis j'ai mis l'api Flask dans un fichier app.py dans le dossier de mon projet. Enfin j'ai lancé l'api Flask avec la commande « python app.py ».

```
saintm@sk3s-01: ~/my_project/flask
saintm@sk3s-01:~/my_project/flask$ ls
app.db  app.py  migrations  __pycache__  venv
saintm@sk3s-01:~/my_project/flask$ cat app.py
import os
import requests
from flask import Flask, jsonify, request
from flask_sqlalchemy import SQLAlchemy
from flask_migrate import Migrate

# Initialisation de l'application Flask
app = Flask(__name__)

# Configuration de la base de données SQLite
basedir = os.path.abspath(os.path.dirname(__file__))
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///' + os.path.join(basedir, 'app.db')
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

# Initialisation de la base de données et des migrations
db = SQLAlchemy(app)
migrate = Migrate(app, db)

# Modèle d'articles
class Article(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(120), nullable=False)

# Endpoint pour récupérer les articles
@app.route('/api/articles', methods=['GET'])
def get_articles():
    articles = Article.query.all()
    articles_list = [{"id": article.id, "title": article.title} for article in articles]
    return jsonify(articles_list)

# Endpoint pour ajouter un nouvel article
```

L'API Flask apparaît au port : http://myproject.local:5000/api/articles.



2. La base de donnée SQLite

SQLite est une base de données relationnelle embarquée utilisée pour assurer la persistance des données (stockage des articles).

Le fichier app.db est créé et stocker dans le dossier de gestion de Flask.

```
saintm@sk3s-01:~/my_project/flask$ cat app.db
SQLite format 3@

  (Mtablearticlearticle CREATE TABLE article (
        id INTEGER NOT NULL,
        title VARCHAR(120) NOT NULL,
        PRIMARY KEY (id)
)  ++  tablealembic_versionalembic_versionCREATE TABLE alembic_version (
        version_num VARCHAR(32) NOT NULL,
        CONSTRAINT alembic_version_pkc PRIMARY KEY (version_num)
  64a6c914d22e_autoindex_alembic_version_1alembic_version
!    W7/mon super articleCmon super troisieme article mon superrrrrrr articleENouvel article depuis Maïss1mon superr article/mon super art
icleCNouvel article depuis SaintANouvel article depuis CURLsaintm@sk3s-01:~/my_project/flask$
```

3. Les notifications WebSocket

Le serveur WebSocket a été implémenté en Go. Il se doit de gérer les connexions WebSocket clients, réagir aux requêtes de notification envoyées par Flask et diffuser en temps réel des messages aux clients connectés dans la console. WebSocket est un protocole de communication permettant à un client et un serveur d'échanger en temps réel c'est utile pour les chats ou encore les notifications par exemple.

Les principales fonctionnalités développées sont, la réception des notifications et leur diffusion en temps réel vers tous les clients connectés, dans le serveur.

On crée un dossier pour le serveur WebSocket, on initialise un module Go et installe Gorilla WebSocket. A l'intérieur de ce dossier, on crée un fichier websocket.go où on insère le code du WebSocket. Enfin on lance le WebSocket avec la commande « go run websocket.go » pour tester.

```
saintm@sk3s-01: ~/my_project/webso
saintm@sk3s-01:~$ cd my_project/
saintm@sk3s-01:~/my_project$ ls
flask  webso
saintm@sk3s-01:~/my_project$ cd webso/
saintm@sk3s-01:~/my_project/webso$ ls
go.mod  go.sum  websocket.go
saintm@sk3s-01:~/my_project/webso$ cat websocket.go
package main

import (
    "log"
    "net/http"
    "github.com/gorilla/websocket"
)

var upgrader = websocket.Upgrader{
    CheckOrigin: func(r *http.Request) bool {
        return true
    },
}

var clients = make(map[*websocket.Conn]bool) // clients WebSocket connectés

func handleWebSocket(w http.ResponseWriter, r *http.Request) {
    conn, err := upgrader.Upgrade(w, r, nil)
    if err != nil {
        log.Println(err)
        return
    }
    defer conn.Close()
    clients[conn] = true

    for {
        _, message, err := conn.ReadMessage()
```

4. Le reverse proxy Nginx

Nginx agit comme reverse proxy dans le projet. Il a été configuré pour rediriger les requêtes HTTP vers le serveur Flask, gérer les connexions WebSocket vers le serveur Go et centraliser les accès via un domaine local (myproject.local). Nginx est un serveur web utilisé pour distribuer des contenus web comme les index HTML et c'est aussi un serveur utilisé pour gérer la charge entre plusieurs services. Le proxy est le serveur intermédiaire entre le client et l'api.

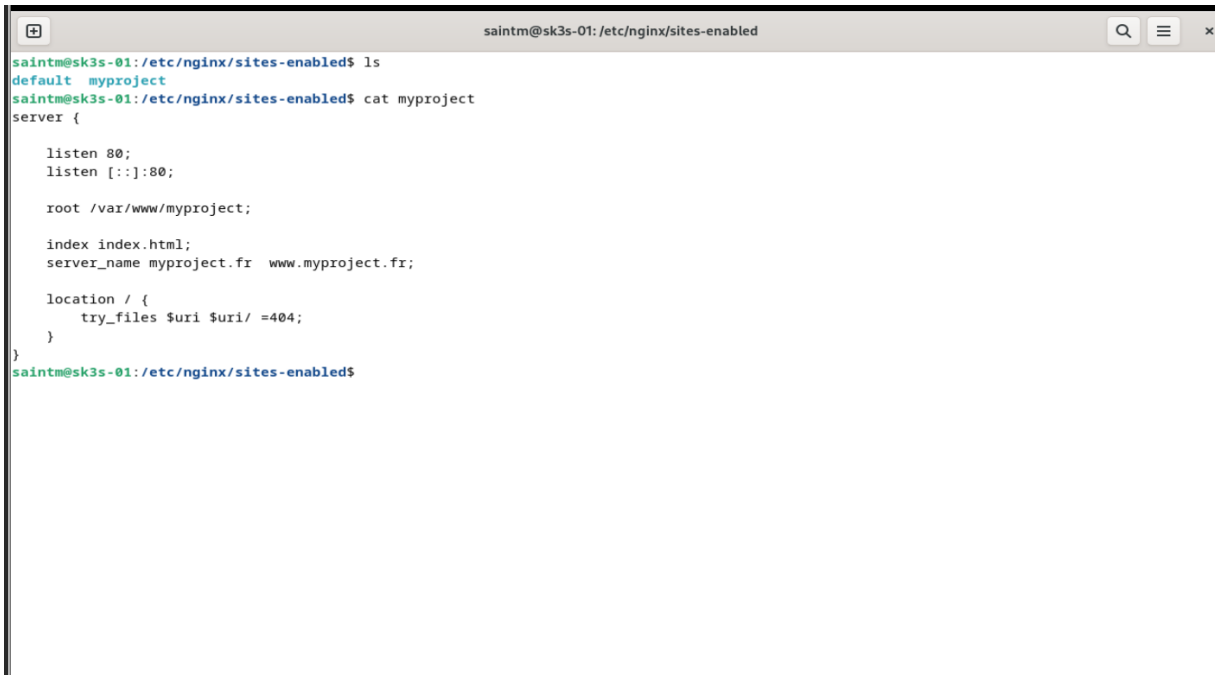
J'ai installé Nginx, l’ai activé pour qu'il soit actif au démarrage de la machine et enfin, j’ai redémarré nginx.

Nginx permet de rediriger les requêtes entrantes vers les bons ports internes où tournent Flask et le serveur Go. Cela permet une exposition unique via le port 80 (HTTP).

J'ai créé un fichier de configuration nginx pour mon projet puis l'ai activé avec un lien symbolique

Le fichier de configuration du site est localisé dans : /etc/nginx/sites-available/myproject

Et lié avec : sudo ln -s /etc/nginx/sites-available/myproject /etc/nginx/sites-enabled/



5. HTML / JavaScript

Une interface utilisateur a été développée en HTML et JavaScript pour afficher dynamiquement la liste des articles, établir une connexion WebSocket avec le serveur et réagir en temps réel aux notifications reçues via WebSocket (mise à jour automatique des articles).

L'interface utilisateur est une page HTML simple, qui utilise JavaScript pour se connecter au serveur WebSocket et afficher dynamiquement les articles.

```
saintm@sk3s-01: /var/www/myproject
saintm@sk3s-01:/var/www$ cd myproject/
saintm@sk3s-01:/var/www/myproject$ ls
index.html
saintm@sk3s-01:/var/www/myproject$ cat index.html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Application Web</title>
</head>
<body>
    <h1>Liste des Articles</h1>
    <ul id="articles"></ul>

    <script>
        // Fonction pour actualiser la liste des articles
        function updateArticles() {
            fetch('/api/articles')
                .then(response => response.json())
                .then(data => {
                    const articlesList = document.getElementById('articles');
                    articlesList.innerHTML = ''; // Efface la liste actuelle
                    data.forEach(article => {
                        let li = document.createElement('li');
                        li.textContent = article.title;
                        articlesList.appendChild(li);
                    });
                });
        }

        // Connexion au WebSocket
        const ws = new WebSocket('ws://myproject.local/ws/');
```

Application Web

myproject.local

Liste des Articles

- Nouvel article depuis CURL
- Nouvel article depuis Saint
- mon super article
- mon superr article
- Nouvel article depuis Maïss
- mon superrrrrrr article
- mon super troisieme article
- mon super article

6. Systemd et ses services

Des fichiers de service systemd ont été utilisés pour automatiser le démarrage et la gestion des processus Flask et Go sur la machine hôte. Cela permet un déploiement propre et une exécution continue des services, même après redémarrage.

Afin d'assurer le bon fonctionnement du projet Multi Webapp, deux services système ont été configurés à l'aide de systemd : un pour le serveur Flask (API), et un autre pour le serveur WebSocket. En complément, Nginx a été mis en place comme reverse proxy pour rediriger les requêtes HTTP vers les bons services.

Les fichiers de configuration des services sont placés dans le répertoire : /etc/systemd/system/

Chacun des fichiers .service permet à systemd de gérer le lancement automatique et la supervision des services.

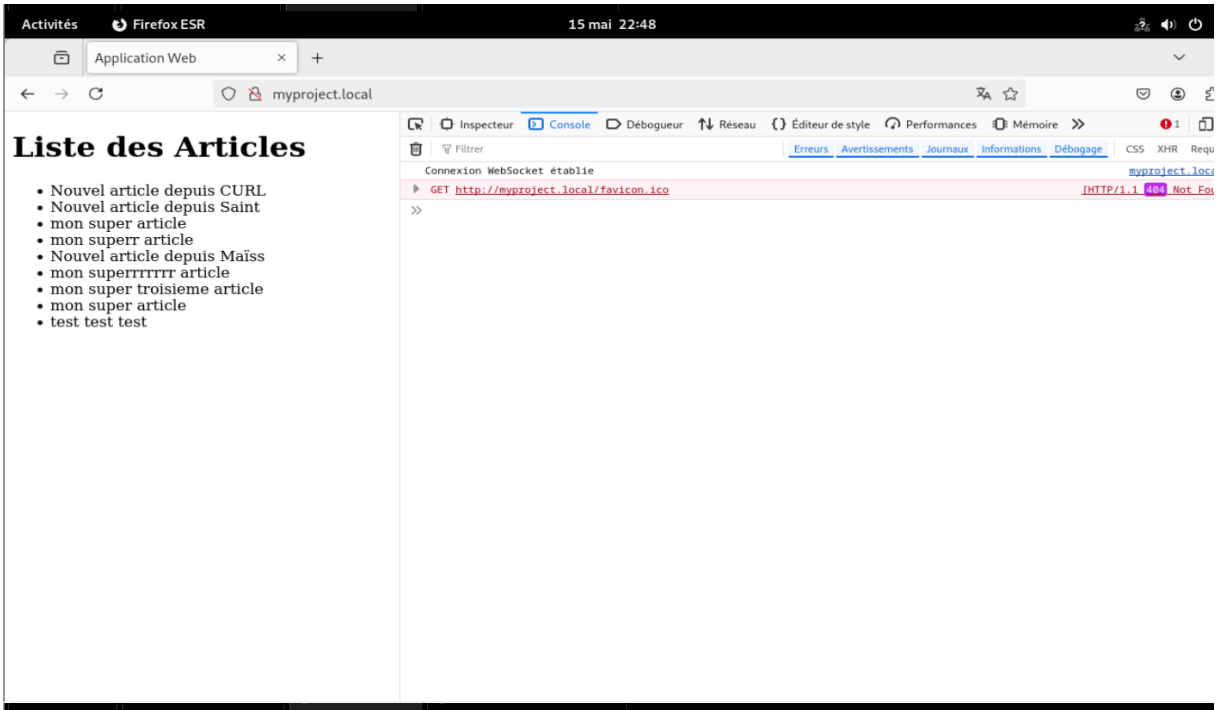
Service Flask & Service WebSocket

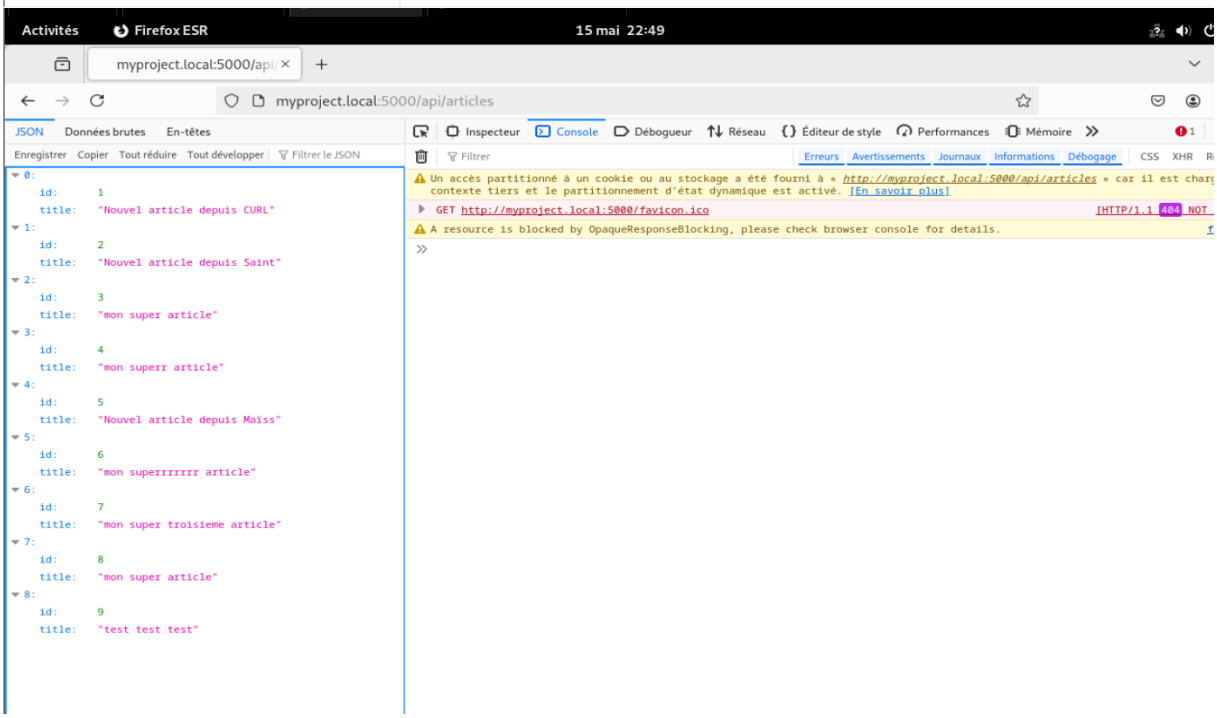
```
saintm@sk3s-01: /etc/systemd/system
bluetooth.target.wants                      flask-app.service              sshd.service
dbus-fi.w1.wpa_supplicant1.service          getty.target.wants             sysinit.target.wants
dbus-org.bluez.service                      graphical.target.wants         timers.target.wants
dbus-org.freedesktop.Avahi.service          multi-user.target.wants        vmtoolsd.service
dbus-org.freedesktop.ModemManager1.service  network-online.target.wants    websocket-app.service
dbus-org.freedesktop.nm-dispatcher.service  open-vm-tools.service.requires
dbus-org.freedesktop.timesync1.service      printer.target.wants
saintm@sk3s-01:/etc/systemd/system$ cat flask-app.service websocket-app.service
[Unit]
Description=Flask Web Application
After=network.target

[Service]
User=saintm
WorkingDirectory=/home/saintm/my_project/flask
Environment="PATH=/home/saintm/my_project/flask/venv/bin"
ExecStart=/home/saintm/my_project/flask/venv/bin/python app.py
Restart=always

[Install]
WantedBy=multi-user.target
[Unit]
Description=Go WebSocket Application
After=network.target

[Service]
User=saintm
WorkingDirectory=/home/saintm/my_project/webso
ExecStart=/usr/local/bin/websocket-app
Restart=always

[Install]
WantedBy=multi-user.target
saintm@sk3s-01:/etc/systemd/system$
```

Après avoir créé les fichiers, les services sont rechargés et configurés pour se lancer à chaque démarrage de la machine.

7. Curl & tests

L'outil en ligne de commande curl a été utilisé pour tester l'API Flask (envoi de requêtes POST) et vérifier le bon fonctionnement de la notification WebSocket.





Activités Terminal 15 mai 22:47

saintm@sk3s-01: ~

```
            └─953 /usr/local/bin/websocket-app

mai 15 08:26:03 sk3s-01 websocket-app[953]: 2025/05/15 08:26:03 Erreur WebSocket: websocket: close 1006 (abnormal closure): unexpected EOF
mai 15 09:52:50 sk3s-01 websocket-app[953]: 2025/05/15 09:52:50 Message reçu : Message depuis le navigateur
mai 15 09:52:51 sk3s-01 websocket-app[953]: 2025/05/15 09:52:51 Erreur WebSocket: websocket: close 1001 (going away)
mai 15 09:52:51 sk3s-01 websocket-app[953]: 2025/05/15 09:52:51 Message reçu : Message depuis le navigateur
mai 15 09:54:08 sk3s-01 websocket-app[953]: 2025/05/15 09:54:08 Erreur WebSocket: websocket: close 1006 (abnormal closure): unexpected EOF
mai 15 22:31:01 sk3s-01 websocket-app[953]: 2025/05/15 22:31:01 Message reçu : Message depuis le navigateur
mai 15 22:31:09 sk3s-01 websocket-app[953]: 2025/05/15 22:31:09 Erreur WebSocket: websocket: close 1001 (going away)
mai 15 22:38:25 sk3s-01 websocket-app[953]: 2025/05/15 22:38:25 Message reçu : Message depuis le navigateur
mai 15 22:39:24 sk3s-01 websocket-app[953]: 2025/05/15 22:39:24 Erreur WebSocket: websocket: close 1006 (abnormal closure): unexpected EOF
mai 15 22:46:28 sk3s-01 websocket-app[953]: 2025/05/15 22:46:28 Notification reçue de Flask
saintm@sk3s-01:~$ systemctl status flask-app.service
● flask-app.service - Flask Web Application
     Loaded: loaded (/etc/systemd/system/flask-app.service; enabled; preset: enabled)
     Active: active (running) since Thu 2025-05-15 09:52:38 CEST; 12h ago
   Main PID: 9806 (python)
      Tasks: 1 (limit: 4579)
     Memory: 50.6M
        CPU: 2.442s
     CGroup: /system.slice/flask-app.service
             └─9806 /home/saintm/my_project/flask/venv/bin/python app.py

mai 15 09:52:39 sk3s-01 python[9806]: Press CTRL+C to quit
mai 15 09:52:50 sk3s-01 python[9806]: 127.0.0.1 - - [15/May/2025 09:52:50] "GET /api/articles HTTP/1.0" 200 -
mai 15 09:52:51 sk3s-01 python[9806]: 127.0.0.1 - - [15/May/2025 09:52:51] "GET /api/articles HTTP/1.0" 200 -
mai 15 22:30:10 sk3s-01 python[9806]: 127.0.0.1 - - [15/May/2025 22:30:10] "GET /api/articles HTTP/1.1" 200 -
mai 15 22:30:11 sk3s-01 python[9806]: 127.0.0.1 - - [15/May/2025 22:30:11] "GET /favicon.ico HTTP/1.1" 404 -
mai 15 22:31:01 sk3s-01 python[9806]: 127.0.0.1 - - [15/May/2025 22:31:01] "GET /api/articles HTTP/1.0" 200 -
mai 15 22:31:09 sk3s-01 python[9806]: 127.0.0.1 - - [15/May/2025 22:31:09] "GET /api/articles HTTP/1.1" 200 -
mai 15 22:38:24 sk3s-01 python[9806]: 127.0.0.1 - - [15/May/2025 22:38:24] "GET /api/articles HTTP/1.0" 200 -
mai 15 22:45:55 sk3s-01 python[9806]: 127.0.0.1 - - [15/May/2025 22:45:55] "POST /api/articles HTTP/1.0" 415 -
mai 15 22:46:28 sk3s-01 python[9806]: 127.0.0.1 - - [15/May/2025 22:46:28] "POST /api/articles HTTP/1.0" 201 -
saintm@sk3s-01:~$
```

Activités Terminal 15 mai 22:46

saintm@sk3s-01: ~

```
saintm@sk3s-01:~$ curl -X POST http://myproject.local/api/articles -H "Content-Type: application/json" -d '{"id": 3, "title": "test test tes
t"}'
{"id":9,"title":"test test test"}
saintm@sk3s-01:~$ systemctl status websocket-app.service
● websocket-app.service - Go WebSocket Application
     Loaded: loaded (/etc/systemd/system/websocket-app.service; enabled; preset: enabled)
     Active: active (running) since Wed 2025-05-14 19:40:57 CEST; 1 day 3h ago
   Main PID: 953 (websocket-app)
      Tasks: 5 (limit: 4579)
     Memory: 15.2M
        CPU: 374ms
     CGroup: /system.slice/websocket-app.service
             └─953 /usr/local/bin/websocket-app

mai 15 08:26:03 sk3s-01 websocket-app[953]: 2025/05/15 08:26:03 Erreur WebSocket: websocket: close 1006 (abnormal closure): unexpected EOF
mai 15 09:52:50 sk3s-01 websocket-app[953]: 2025/05/15 09:52:50 Message reçu : Message depuis le navigateur
mai 15 09:52:51 sk3s-01 websocket-app[953]: 2025/05/15 09:52:51 Erreur WebSocket: websocket: close 1001 (going away)
mai 15 09:52:51 sk3s-01 websocket-app[953]: 2025/05/15 09:52:51 Message reçu : Message depuis le navigateur
mai 15 09:54:08 sk3s-01 websocket-app[953]: 2025/05/15 09:54:08 Erreur WebSocket: websocket: close 1006 (abnormal closure): unexpected EOF
mai 15 22:31:01 sk3s-01 websocket-app[953]: 2025/05/15 22:31:01 Message reçu : Message depuis le navigateur
mai 15 22:31:09 sk3s-01 websocket-app[953]: 2025/05/15 22:31:09 Erreur WebSocket: websocket: close 1001 (going away)
mai 15 22:38:25 sk3s-01 websocket-app[953]: 2025/05/15 22:38:25 Message reçu : Message depuis le navigateur
mai 15 22:39:24 sk3s-01 websocket-app[953]: 2025/05/15 22:39:24 Erreur WebSocket: websocket: close 1006 (abnormal closure): unexpected EOF
mai 15 22:46:28 sk3s-01 websocket-app[953]: 2025/05/15 22:46:28 Notification reçue de Flask
saintm@sk3s-01:~$
```